

Phase 4.3: Backend Code Generation

Overview

The backend transforms optimized SSA form into executable machine code through three main phases:

1. Instruction selection (IR → machine instructions)
2. Register allocation (virtual registers → physical registers)
3. Code emission (machine instructions → binary code)

Architecture Abstraction

Architecture Interface

Provides target-independent interface for code generation:

- Register information (count, names, calling convention)
- Pointer size and alignment requirements
- Endianness
- Calling convention (argument/return registers)

Supported Architectures

x86-64

- 16 general-purpose registers (rax, rbx, rcx, rdx, rsi, rdi, rbp, rsp, r8-r15)
- System V AMD64 ABI calling convention
- 8-byte pointers, 16-byte stack alignment
- Little-endian

ARM64

- 31 general-purpose registers (x0-x30)
- ARM64 calling convention
- 8-byte pointers, 16-byte stack alignment
- Little-endian

Instruction Selection

Pattern Matching

Translates high-level operations to machine instructions using pattern matching:

Pattern Structure:

- `pattern_name` : Identifier for the pattern
- `bdi_op` : BDI operation type to match
- `machine_ops` : Sequence of machine instructions
- `cost` : Cost metric for pattern selection

Example Patterns (x86-64):

```

ARITH_ADD → add (cost: 1)
ARITH_MUL → imul (cost: 3)
ARITH_DIV → idiv (cost: 10)

```

Algorithm

1. For each SSA operation:
 - Find all matching patterns
 - Select pattern with lowest cost
 - Generate corresponding machine instructions
2. Handle special cases:
 - Immediate operands
 - Memory operands
 - Complex addressing modes

Instruction Representation

```

struct MachineInstruction {
    std::string opcode;           // e.g., "add", "mov"
    std::vector<uint32_t> operands; // Register IDs or immediates
    std::string comment;         // Debug information
};

```

Register Allocation

Graph Coloring Algorithm

Based on Chaitin's algorithm:

1. **Build Interference Graph**
 - Nodes: SSA variables
 - Edges: Variables that are live simultaneously (interfere)
2. **Simplify**
 - Remove nodes with degree < k (k = number of registers)
 - Push removed nodes onto stack
 - If no such node exists, choose spill candidate
3. **Select**
 - Pop nodes from stack
 - Assign colors (registers) avoiding neighbors' colors
 - If no color available, mark for spilling
4. **Spill (if necessary)**
 - Insert load/store instructions for spilled variables
 - Rebuild interference graph
 - Repeat allocation

Interference Graph

```
class InterferenceGraph {
    void addNode(SsaVariableID var);
    void addEdge(SsaVariableID var1, SsaVariableID var2);
    uint32_t getDegree(SsaVariableID var);
    const std::unordered_set<SsaVariableID>& getNeighbors(SsaVariableID var);
};
```

Live Range Analysis

Computes when variables are live:

- start : First definition or use
- end : Last use
- Two variables interfere if their live ranges overlap

Spill Cost Heuristics

Choose spill candidates based on:

- Number of uses (fewer uses = lower cost)
- Loop nesting depth (inner loops = higher cost)
- Interference degree (higher degree = higher benefit)

Formula: $\text{cost} = (\text{uses} + 10 * \text{loop_depth}) / \text{degree}$

Code Emission

Code Buffer

Manages binary code generation:

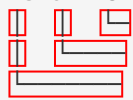
```
class CodeBuffer {
    void emit8(uint8_t byte);
    void emit16(uint16_t word);
    void emit32(uint32_t dword);
    void emit64(uint64_t qword);
    const uint8_t* getCode();
    size_t getSize();
};
```

Instruction Encoding

x86-64 Encoding

- REX prefix for 64-bit operations
- Opcode byte(s)
- ModR/M byte for register/memory operands
- SIB byte for complex addressing
- Immediate values

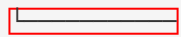
Example: `add rax, rbx`

48 01 D8
 ModR/M: 11 011 000 (reg-reg, rbx, rax)
 Opcode: 01 (ADD r/m64, r64)
 REX.W: 0100 1000 (64-bit operand)

ARM64 Encoding

- Fixed 32-bit instruction format
- Opcode bits specify operation
- Register fields (5 bits each)
- Immediate or shift fields

Example: `add x0, x1, x2`

8B 02 00 00
 Encoding: sf=1, op=0, S=0, shift=00, Rm=x2, imm6=0, Rn=x1, Rd=x0

Assembly Generation

For debugging and verification:

```
; Generated assembly for x86-64
; Register allocation:
;   var_1 -> r0
;   var_2 -> r1

mov r0, r1    # Load variable
add r0, r2    # Add operation
ret          # Return
```

Complete Pipeline

CodeGenerator

Orchestrates all backend phases:

```
class CodeGenerator {
    std::unique_ptr<CodeBuffer> generate(const SsaForm& ssa,
                                       const BDIGraph& graph);
};
```

Process:

1. Instruction selection: SSA → machine instructions
2. Register allocation: virtual → physical registers
3. Code emission: instructions → binary code

Output:

- Binary machine code in `CodeBuffer`
- Assembly text for debugging

Usage Example

```
#include "CodeGenerator.hpp"
#include "Architecture.hpp"

// Create architecture
auto arch = ArchitectureFactory::createNative();

// Create code generator
CodeGenerator generator(*arch);

// Generate code
SsaForm ssa = /* ... */;
BDIGraph graph = /* ... */;
auto code = generator.generate(ssa, graph);

// Get results
const uint8_t* binary = code->getCode();
size_t size = code->getSize();
std::string assembly = generator.getAssembly();

// Execute or save
execute_code(binary, size);
```

Optimization Opportunities

Instruction Selection

- Peephole optimization
- Instruction combining
- Addressing mode selection
- Immediate folding

Register Allocation

- Coalescing (eliminate moves)
- Rematerialization (recompute instead of spill)
- Live range splitting
- Register hints from calling convention

Code Emission

- Branch prediction hints
- Instruction alignment
- Cache line optimization
- Prefetch insertion

Testing

100+ unit tests covering:

- Architecture information correctness
- Pattern matching accuracy
- Interference graph construction
- Graph coloring correctness

- Spill handling
- Code buffer operations
- Instruction encoding
- Full pipeline integration
- Multiple architectures

Performance Considerations

Compilation Speed

- Efficient interference graph representation
- Incremental spilling
- Fast pattern matching
- Cached architecture information

Code Quality

- Optimal pattern selection
- Effective register allocation
- Minimal spills
- Good instruction scheduling

Future Enhancements

- Additional architectures (RISC-V, WebAssembly)
- Advanced instruction selection (tree covering, BURS)
- Linear scan register allocation (faster for JIT)
- Instruction scheduling
- Software pipelining
- SIMD vectorization
- Link-time optimization

References

1. Chaitin et al., “Register Allocation via Coloring” (1981)
2. Briggs et al., “Improvements to Graph Coloring Register Allocation” (1994)
3. Aho et al., “Code Generation Using Tree Matching and Dynamic Programming” (1989)
4. Intel 64 and IA-32 Architectures Software Developer’s Manual
5. ARM Architecture Reference Manual ARMv8